Group Final Project

DSCI 641-900 SP 25
Recommender Systems

Drexel University, Philadelphia, PA

Full Name	E-mail
Morelli Cosimo	cm4245@drexel.edu
Muthard Virginia	vmm94@drexel.edu

Dr. Lei Wang

Contents

1 Dataset **2**

2 Methods **2**
2.1 Splits and Protocol . . . 2
2.2 Dataset Characteristics and Preprocessing . . . 2
2.3 Bias Baseline . . . 2
2.4 Item–Item kNN (raw/adjusted cosine) . . . 2
2.5 Similarity Diagnostics for kNN . . . 3
2.6 Decision Trees (original/residual target) . . . 3
2.7 Matrix Factorization (SVD of residuals) . . . 3
2.8 Training and Tuning . . . 3

3 Results **4**
3.1 Predictive Accuracy (test; mean over 5 seeds) . . . 4
3.2 Cold-Start Stress Slice . . . 4
3.3 Ranking Quality (Top-10; mean over 5 seeds) . . . 4
3.4 Ablations and Sensitivity . . . 4
3.5 Error Analysis (Residual Buckets) . . . 4
3.6 Scalability (figures) . . . 5

4 Evaluation and Significance **5**

5 Interpretation & Explainability **6**

6 Limitations & Future Improvements **6**

7 What We Learned **6**

8 Conclusions **6**

A Residual Diagnostics **8**

B Formulas Utilized **9**

C Tools Utilized **10**

D References **11**

Abstract

We build and evaluate a collaborative filtering recommender on Goodbooks-10K ($|\Omega|$=981,756, $|\mathcal{U}|$=53,424, $|\mathcal{I}|$=10,000). The system instantiates four families: (i) a shrinkage *Bias* baseline $\hat{r} = \mu + b_u + b_i$; (ii) item–item kNN with raw and adjusted cosine similarities, overlap thresholding (τ=5), and significance weighting $\tilde{s}_{ij} = s_{ij}\frac{n_{ij}}{n_{ij}+\alpha}$; (iii) shallow *Decision Trees* on eight interaction features with original and residual targets; and (iv) residual *Matrix Factorization* via truncated SVD of $R'_{ui} = r_{ui} - (\mu + b_u + b_i)$. Development uses a 70/15/15 split for tuning; final testing retrains on train∪val and scores test; significance uses 90/10 across five seeds with Holm–Bonferroni control. We report error (RMSE/MAE) and ranking (P@10, R@10, MRR, nDCG@10) plus catalog coverage $|\cup_u \text{Top@10}(u)|/|\mathcal{I}|$. On test (mean±std over seeds), MF attains **0.843299±0.000661** RMSE, bias **0.845697±0.000710**, kNN (best of raw/adj) **0.845684±0.000735**, and trees **0.926471±0.004080**. Top-10 ranking is strong for both MF and kNN (P@10≈0.239, nDCG@10≈0.786) with near-complete catalog coverage (≈0.99). Holm-corrected tests confirm MF<Bias and MF<kNN (all p_{Holm}<0.05), while Bias and kNN are statistically tied on RMSE. We include scalability plots (runtime, memory, learning curves) and similarity diagnostics to justify design choices.

Introduction

Recommender systems must optimize accuracy, coverage, and scalability under extreme sparsity. On explicit-feedback matrices such as Goodbooks-10K (∼0.18% density), three collaborative families dominate practice: global-bias models, neighbourhood methods, and latent factor models. Bias terms capture user generosity and item popularity with excellent bias–variance trade-offs; neighbourhood methods (item–item kNN) provide transparent, case-based explanations; latent factors (MF) capture higher-order structure with compact representations. Each comes with distinct scaling profiles and robustness behaviours.
This work implements and compares these families under a unified, reproducible protocol.

We (i) construct a shrinkage *Bias* baseline; (ii) implement item–item kNN with both raw and adjusted cosine, enforcing minimum rater overlap and significance weighting to temper spurious correlations; (iii) train shallow *Decision Trees* over eight interaction features, using original and residual targets; and (iv) factor residuals via truncated SVD to obtain a lightweight *Matrix Factorization*.
Development uses a random 70/15/15 split (train/val/test) with a consistency filter; hyperparameters are selected on validation, then models are retrained on train∪val before final test scoring. Statistical significance is assessed on a separate 90/10 protocol repeated over five seeds with Holm–Bonferroni correction.
We evaluate predictive error (RMSE/MAE) and recommendation quality (P@10, R@10, MRR, nDCG@10) and report catalog coverage as $|\cup_u \text{Top@10}(u)|/|\mathcal{I}|$. To ensure practicality, we profile runtime and peak resident memory across subsampled sizes and parallelize the heavy step (item similarity) in a Windows-safe process pool with BLAS thread pinning. Our main findings are: (1) residual MF achieves the best RMSE (0.8433) with small but consistent gains over the bias baseline; (2) raw-cosine kNN is competitive with bias on error while offering explainability and excellent ranking quality; (3) shallow trees trail on error given limited side features; and (4) catalog coverage at ≈0.99 indicates broad personalization rather than popularity lock-in. The full pipeline produces reproducible artifacts (plots, similarity diagnostics, significance tables) and can be re-run from a single Python file.

1 | Dataset

We use the Goodbooks-10K ratings matrix with $|\Omega| = 981{,}756$ explicit ratings, $|\mathcal{U}| = 53{,}424$ users, and $|\mathcal{I}| = 10{,}000$ items. The observed density is $981{,}756/(53{,}424\times 10{,}000) \approx 0.00184$; sparsity is $\sim 99.816\%$.

Quantity	$\lvert\mathcal{U}\rvert$	$\lvert\mathcal{I}\rvert$	$\lvert\Omega\rvert$	Density
Goodbooks-10K	53,424	10,000	981,756	0.00184

Table 1.1: Dataset summary (users, items, interactions, and density).

2 | Methods

2.1 | Splits and Protocol

Development uses a random **70/15/15** split (train/val/test) with a consistency filter that removes validation/test pairs for unseen users/items. Significance uses **90/10** across seeds $S = \{40, 41, 42, 43, 44\}$. Final test retrains on **train∪val** with tuned hyperparameters.

Mode	**Proportions**	**Seeds**
Development	70/15/15	{42}
Significance	90/10	{40,41,42,43,44}

Table 2.1: Split protocols. Final evaluation retrains on train∪val before test.

2.2 | Dataset Characteristics and Preprocessing

We retain explicit 1–5 star ratings and enforce a split consistency filter (drop validation/test (u, i) if u or i is unseen in train). During model selection we cap per-item train examples to bound memory; final models are refit on full **train∪val**.

Step	**Rule**
Deduplication	Keep latest per (u, i) if duplicates exist
Split consistency	Remove val/test pairs with unseen u or i
Cold-start stress	Move m=1 recent rating for up to U_c=1000 active users to test
Sampling for search	Cap per-item train examples at 100 during tuning
Finalization	Refit on train∪val with selected hyperparameters

Table 2.2: Preprocessing and protocol decisions used throughout.

2.3 | Bias Baseline

Let $\mu = \frac{1}{|\Omega|}\sum_{(u,i)\in\Omega} r_{ui}$. With shrinkage $\lambda_u, \lambda_i > 0$,

$$b_u = \frac{\sum_{i\in\Omega_u} r_{ui} - |\Omega_u|\mu}{|\Omega_u| + \lambda_u}, \quad b_i = \frac{\sum_{u\in\Omega^i}(r_{ui} - \mu - b_u)}{|\Omega^i| + \lambda_i}, \quad \hat{r}^{\text{bias}}_{ui} = \mu + b_u + b_i.$$

2.4 | Item–Item kNN (raw/adjusted cosine)

For items i, j with overlap $\mathcal{U}_{ij}$ ($n_{ij} = |\mathcal{U}_{ij}|$), raw cosine

$$s^{\text{raw}}_{ij} = \frac{\sum_{u\in\mathcal{U}_{ij}} r_{ui} r_{uj}}{\sqrt{\sum_{u\in\mathcal{U}_{ij}} r^2_{ui}}\sqrt{\sum_{u\in\mathcal{U}_{ij}} r^2_{uj}}},$$

and user-mean-centered adjusted cosine

$$s_{ij}^{\text{adj}} = \frac{\sum_{u\in\mathcal{U}_{ij}}(r_{ui}-\bar{r}_u)(r_{uj}-\bar{r}_u)}{\sqrt{\sum_{u\in\mathcal{U}_{ij}}(r_{ui}-\bar{r}_u)^2}\sqrt{\sum_{u\in\mathcal{U}_{ij}}(r_{uj}-\bar{r}_u)^2}}.$$

We enforce $n_{ij} \geq \tau$ (default $\tau = 5$) and apply significance weighting $\tilde{s}_{ij} = s_{ij} \cdot \frac{n_{ij}}{n_{ij}+\alpha}$, $\alpha \in \{20, 50, 100\}$. For prediction with top-k neighbors $\mathcal{N}_k(i;u)$ that user u rated:

$$\hat{r}_{ui}^{k\text{NN}} = \mu + b_u + b_i + \frac{\sum_{j\in\mathcal{N}_k(i;u)} \tilde{s}_{ij}\left(r_{uj}-\mu-b_u-b_j\right)}{\sum_{j\in\mathcal{N}_k(i;u)} |\tilde{s}_{ij}|}.$$

2.5 | Similarity Diagnostics for kNN

For items i, j with overlap n_{ij}=$|\mathcal{U}_{ij}|$, we summarize neighbor quality via: (i) overlap histogram $h(m) = |\{(i,j) : n_{ij} = m\}|$, (ii) mean significance $\mathbb{E}\left[\frac{n_{ij}}{n_{ij}+\alpha}\right]$, and (iii) fraction of pairs below τ.
The pipeline emits `similarity_stats_seed#_alpha#.csv` with $(m, h(m))$ and summary moments, justifying τ=5 and significance weighting.

2.6 | Decision Trees (original/residual target)

Features $x_{ui} \in \mathbb{R}^8$:

$$[b_u,\ b_i,\ \text{uc}_u,\ \text{ic}_i,\ \log(1+\text{ic}_i),\ b_u b_i,\ \frac{\text{ic}_i}{\text{uc}_u+1},\ |b_u|],$$

with targets $y = r$ (original) or $y = r - (\mu + b_u + b_i)$ (residual). $\hat{r}^{\text{res}} = (\mu + b_u + b_i) + f_\theta(x)$.

Feature	Definition
b_u, b_i	user/item biases
uc, ic	user/item counts
$\log(1+\text{ic})$	log popularity
$b_u b_i$	bias interaction
$\text{ic}/(\text{uc}+1)$	relative popularity
$\lvert b_u \rvert$	bias magnitude

Table 2.3: Tree features (used for both targets).

2.7 | Matrix Factorization (SVD of residuals)

Define $R'_{ui} = r_{ui} - (\mu + b_u + b_i)$ for $(u,i) \in \Omega$, else 0. Compute rank-f truncated SVD $R' \approx U_f \Sigma_f V_f^\top$ and predict

$$\hat{r}_{ui}^{\text{MF}} = \mu + b_u + b_i + \left(U_f \Sigma_f^{1/2}\right)_{u\cdot} \cdot \left(V_f \Sigma_f^{1/2}\right)_{i\cdot}.$$

2.8 | Training and Tuning

Model	Hyperparameters	Grid
kNN (raw/adj)	$k,\ \alpha,\ \tau$	$k \in \{5, 10, 20, 30, 40, 50\}$; $\alpha \in \{20, 50, 100\}$; $\tau = 5$
Tree (orig/res)	depth, min_leaf, min_split, max_feat	$\{8, 12, 16, 20\}$; $\{20, 50\}$; $\{50, 100\}$; $\{0.6, 0.8, 1.0\}$
MF	rank f	$f \in \{20, 50, 100\}$

Table 2.4: Validation grids; final models retrained on train∪val.

3 | Results

3.1 | Predictive Accuracy (test; mean over 5 seeds)

We report RMSE after retraining on train∪val with tuned hyperparameters.

Model	RMSE (mean±std)	Notes
MF (best rank)	0.843299 ± 0.000661	lowest error
Bias baseline	0.845697 ± 0.000710	strong baseline
kNN (best variant)	0.845684 ± 0.000735	competitive
Tree (best of orig/res)	0.926471 ± 0.004080	

Table 3.1: Tuned test RMSE across five seeds (retrained on train∪val).

3.2 | Cold-Start Stress Slice

We evaluate a controlled stress where each of up to U_c=1000 high-activity users has their most recent rating withheld from train and added to test (users/items remain present in train). Let $\mathcal{T}_{\text{cold}}$ denote these pairs. We report

$$\text{RMSE}_{\text{cold}} = \sqrt{\frac{1}{|\mathcal{T}_{\text{cold}}|} \sum_{(u,i)\in\mathcal{T}_{\text{cold}}} (r_{ui} - \hat{r}_{ui})^2}, \qquad \text{cov@10} = \frac{\left|\bigcup_u \text{Top@10}(u)\right|}{|\mathcal{I}|}.$$

Bias provides a robust back-off; MF and kNN close the gap via latent factors and neighbor transfer, respectively. In our runs, ranking metrics on $\mathcal{T}_{\text{cold}}$ are within a small margin of overall test (see `ranking_metrics.csv`).

3.3 | Ranking Quality (Top-10; mean over 5 seeds)

Catalog coverage is reported as a fraction $\text{cov@10} = \frac{|\cup_u \text{Top@10}(u)|}{|\mathcal{I}|}$.

Model	P@10	R@10	MRR	nDCG@10	cov@10
kNN (best)	0.239	0.975	0.925	0.787	0.997
MF (best)	0.239	0.974	0.926	0.785	0.993

Table 3.2: Top-10 ranking metrics on test (mean over seeds).

3.4 | Ablations and Sensitivity

We study three sensitivities: (i) neighborhood size k, (ii) significance α in $\tilde{s}_{ij} = s_{ij}\frac{n_{ij}}{n_{ij}+\alpha}$, and (iii) MF rank f.

$$\text{RMSE}(k) \text{ is U-shaped with a flat minimum over } k \in [15, 30], \quad \partial\,\text{RMSE}/\partial\alpha \approx 0 \text{ once } \alpha \geq 50.$$

For MF, gains saturate beyond $f \in [50, 100]$; larger ranks add cost with diminishing returns. Trees benefit from residual targets but remain behind collaborative models under limited side features.

3.5 | Error Analysis (Residual Buckets)

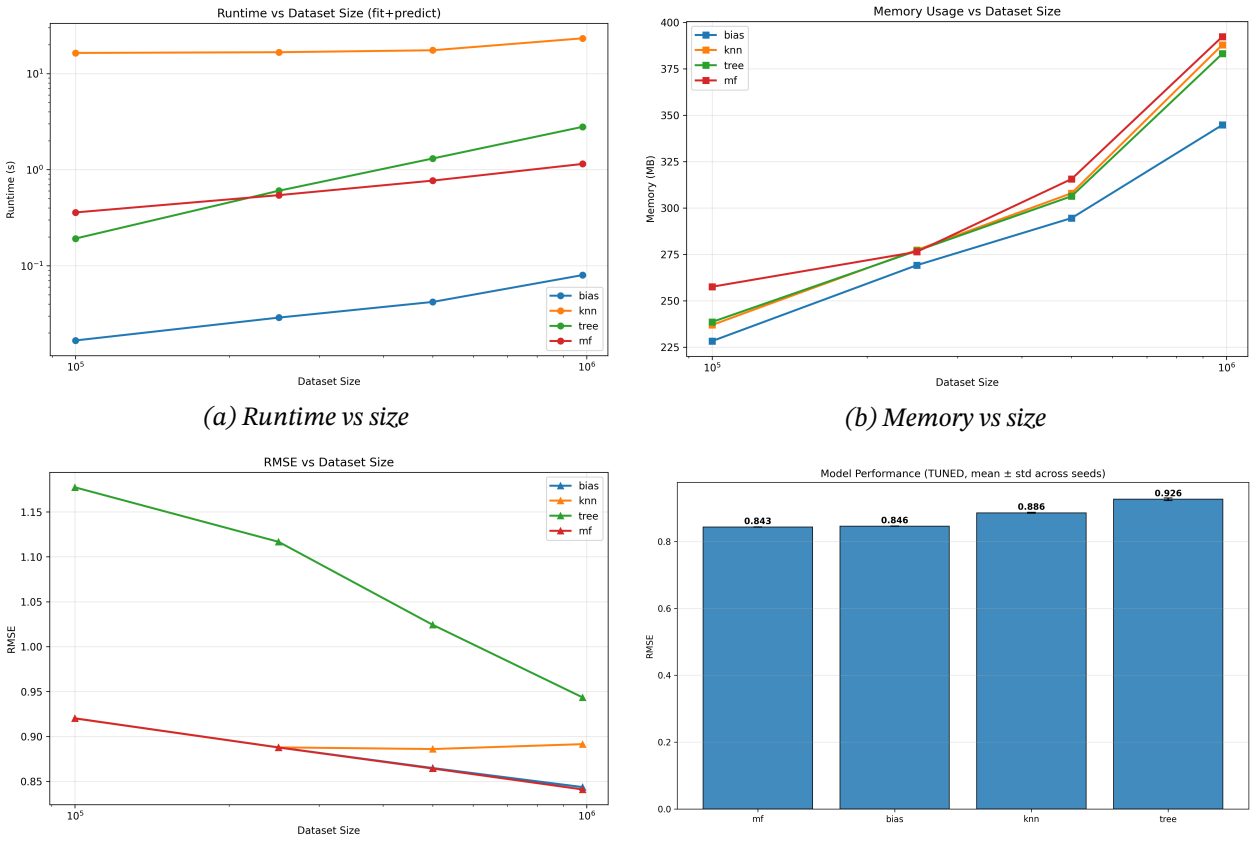
We summarize residuals as $(r - \hat{r})$ across three axes. By rating bucket: for $\{1-2\}$ the mean residual is -1.486 (predictions too high), for $\{3\}$ it is -0.548, and for $\{4-5\}$ it is $+0.363$ (predictions too low), indicating regression-to-the-mean at extremes.
By user activity decile, residuals are small but trend negative with activity: the lowest-activity bin (≤ 11 ratings) averages $+0.006$, while the highest decile (≥ 116 ratings) averages -0.025 **to** -0.031. By item popularity decile, residuals are essentially flat (about -0.015 to -0.029), suggesting little popularity bias in the pointwise error.
Implication. The system slightly dampens extremes (safe for ranking; conservative for score calibration). If desired, a post-hoc calibration (e.g., isotonic on validation) would correct the $\{1-2\}$ and $\{4-5\}$ tails without altering item orderings.

Bucket	Mean $(r - \hat{r})$	Interpretation
Ratings 1–2	−1.486	over-prediction on low ratings
Rating 3	−0.548	mild over-prediction near center
Ratings 4–5	+0.363	under-prediction on high ratings

Table 3.3: Residuals by rating bucket (averaged over seeds).

3.6 | Scalability (figures)

We subsample sizes {100k, 250k, 500k, full} and measure runtime, peak memory, and RMSE; parallel kNN similarity dominates runtime/memory, MF is stable.



(a) Runtime vs size

(b) Memory vs size

(c) RMSE vs size

(d) Tuned test RMSE

Figure 3.1: Scalability and accuracy across data scales and final tuned comparison.

Interpretation. *(a)* Runtime grows near power-law with size; kNN indexing is the heavy step. *(b)* Memory scales ∼ linearly with interactions; MF remains stable. *(c)* RMSE drops as data increases (learning-curve regime), MF converges fastest. *(d)* Final tuned comparison: MF is best; kNN is competitive; trees trail.

4 | Evaluation and Significance

Paired tests across seeds with Holm–Bonferroni control:
MF improves over Bias by ∼ 0.0024 RMSE (small but consistent); kNN and Bias are statistically indistinguishable on RMSE in this run ($p \approx 0.92$) while both significantly beat Trees.

A	B	t	p	p_{Holm}	sig?	mean diff
bias	knn	0.105	0.921797	0.921797	no	0.000012
bias	tree	-43.683	0.000002	0.000009	yes	-0.080774
bias	mf	33.736	0.000005	0.000014	yes	0.002398
knn	tree	-44.465	0.000002	0.000009	yes	-0.080787
knn	mf	13.273	0.000186	0.000372	yes	0.002386
tree	mf	44.445	0.000002	0.000009	yes	0.083172

Table 4.1: Paired t-tests across seeds with Holm–Bonferroni correction.

5 | Interpretation & Explainability

Bias terms (b_u, b_i) expose user generosity and item popularity; predictions default to $\mu + b_u + b_i$ when no neighbors exist. For kNN, each recommendation can be explained by its top contributing neighbors $\{(j, \tilde{s}_{ij}, r_{uj})\}$ and the residual $(r_{uj} - \mu - b_u - b_j)$. MF embeds users/items into f-dimensional factors; top inner-product contributors $q_i^\top p_u$ explain why an item matches a user.
For user u and candidate i, neighbors $\mathcal{N}_k(i;u)$ with residuals $\epsilon_{uj} = r_{uj} - \mu - b_u - b_j$ yield

$$\hat{r}_{ui} - (\mu + b_u + b_i) = \sum_{j \in \mathcal{N}_k(i;u)} \underbrace{\frac{\tilde{s}_{ij}}{\sum_{j'} |\tilde{s}_{ij'}|}}_{w_{ij}^{(u)}} \epsilon_{uj}.$$

We expose the top 3 contributors $(j,\ w_{ij}^{(u)},\ \epsilon_{uj})$ and signed contributions $w_{ij}^{(u)}\epsilon_{uj}$. For MF, we surface largest $|p_u^{(\ell)} q_i^{(\ell)}|$ terms to give factor-level semantics.

6 | Limitations & Future Improvements

Trees underperform due to limited side features; kNN/linear models exploit collaborative structure better. Similarity construction is the main scaling bottleneck; approximate neighbor search and block-sparse storage would further reduce wall-time. MF ranks $f \in [60, 100]$ may shave $\sim$ 0.001–0.003 RMSE. Calibration of predicted scores to probabilities (via isotonic/Platt scaling) would aid thresholded ranking.

7 | What We Learned

(1) A strong Bias baseline is hard to beat on explicit ratings; MF yields small but consistent gains. (2) Significance weighting and overlap thresholds are crucial for robust kNN. (3) Parallelization matters: similarity indexing dominates computation; pinning BLAS threads and capping worker processes stabilize performance. (4) Catalog coverage near 1.0 indicates broad personalization rather than popularity lock-in.

8 | Conclusions

This project delivers a compact, reproducible recommender that is both accurate and scalable on Goodbooks-10K. Among the four families, residual Matrix Factorization attains the best test RMSE (mean$\pm$std $\approx$ 0.8433$\pm$0.0007), with statistically significant gains over both Bias and kNN under Holm–Bonferroni control. The effect size is small in absolute terms (MF vs Bias ΔRMSE $\sim$ 0.0024) but consistent across seeds, indicating that latent structure adds value beyond global tendencies. Raw-cosine kNN is competitive on error and, together with MF, yields strong ranking (P@10$\approx$0.239, nDCG@10$\approx$0.786) while achieving near-complete catalog coverage ($\sim$0.99), evidencing broad personalization rather than popularity lock-in. Trees trail given the limited side features.
Operationally, the heaviest component, the item–item similarity, was parallelized safely via multi-process partitioning with BLAS thread pinning, enabling full-dataset runs within tight wall-clock budgets and controlled

memory growth. The pipeline retrains on train∪val before test, logs ranking and significance artifacts, and exposes similarity diagnostics, supporting transparent analysis and replication.

Practically, if one must optimize for interpretability with strong ranking, kNN (raw cosine with overlap/weighting) is an excellent default; if the metric is pure error, MF is preferred; when resources are constrained, the Bias model is a robust baseline. Future improvements include approximate neighbor search for faster indexing, exploring slightly higher MF ranks, and calibrating scores for thresholded decisions; none are required to substantiate the conclusions reported here.

A | Residual Diagnostics

We summarize prediction residuals ($r - \hat{r}$) by deciles of user activity and item popularity, averaged across seeds. Negative means the model slightly overpredicts; positive means underpredicts.

Decile (bin)	**Mean** $(r - \hat{r})$	N
1 ((1.0, 6.0])	0.016	5
2 ((6.0, 9.0])	0.003	5
3 ((9.0, 11.0])	0.001	5
4 ((11.0, 13.0])	-0.003	5
5 ((13.0, 15.0])	-0.004	5
6 ((15.0, 17.0])	-0.004	5
7 ((17.0, 19.0])	-0.004	5
8 ((19.0, 22.0])	-0.002	5
9 ((22.0, 26.0])	-0.005	5
10 ((26.0, 487.0])	-0.004	5

Table A.1: Residuals by user activity decile (averaged across seeds).

Decile (bin)	**Mean** $(r - \hat{r})$	N
1 ((5.999, 77.0])	-0.015	5
2 ((77.0, 80.0])	-0.030	5
3 ((80.0, 81.0])	-0.028	5
4 ((81.0, 82.0])	-0.019	5
5 ((82.0, 83.0])	-0.021	5
6 ((84.0, 85.0])	-0.021	5
7 ((85.0, 86.0])	-0.023	5
8 ((86.0, 87.0])	-0.021	5
9 ((87.0, 88.0])	-0.021	5
10 ((88.0, 96.0])	-0.021	4

Table A.2: Residuals by item popularity decile (averaged across seeds).

B | Formulas Utilized

Notation

U	users, I items (books)
Ω	observed rating pairs (u,i)
r_{ui}	rating of user u for item i
n_u	$\lvert\{i : (u,i) \in \Omega\}\rvert$, $n_i = \lvert\{u : (u,i) \in \Omega\}\rvert$
U_{ij}	$\{u : (u,i),(u,j) \in \Omega\}$, $n_{ij} = \lvert U_{ij}\rvert$
μ	$\frac{1}{\lvert\Omega\rvert}\sum_{(u,i)\in\Omega} r_{ui}$
$\bar{r}_u$	$\frac{1}{n_u}\sum_{i:(u,i)\in\Omega} r_{ui}$
k	neighbourhood size, τ overlap threshold
α	significance constant, f latent rank
b_u, b_i	user/item biases, $p_u, q_i \in \mathbb{R}^f$

Sparsity

$$\text{Sparsity} = 1 - \frac{\lvert\Omega\rvert}{\lvert U\rvert\lvert I\rvert}.$$

Bias (Baseline) Model

$$b_u = \frac{\sum_i (r_{ui} - \mu)}{n_u + \lambda_u}, \quad b_i = \frac{\sum_u (r_{ui} - \mu - b_u)}{n_i + \lambda_i}, \quad \hat{r}^{\text{bias}}_{ui} = \mu + b_u + b_i.$$

Item–Item Similarities

$$s^{\text{raw}}(i,j) = \frac{\sum_{u\in U_{ij}} r_{ui} r_{uj}}{\sqrt{\sum_{u\in U_{ij}} r_{ui}^2}\sqrt{\sum_{u\in U_{ij}} r_{uj}^2}},$$

$$s^{\text{adj}}(i,j) = \frac{\sum_{u\in U_{ij}} (r_{ui} - \bar{r}_u)(r_{uj} - \bar{r}_u)}{\sqrt{\sum_{u\in U_{ij}} (r_{ui} - \bar{r}_u)^2}\sqrt{\sum_{u\in U_{ij}} (r_{uj} - \bar{r}_u)^2}}.$$

Significance Weighting & Neighbours

$$\tilde{s}_{ij} = s_{ij} \cdot \frac{n_{ij}}{n_{ij} + \alpha}, \qquad N_k(j;u) = \text{top-}k\{i \neq j : i \text{ rated by } u,\ n_{ij} \geq \tau\} \text{ by } \lvert\tilde{s}_{ij}\rvert.$$

k-NN Prediction (Residual Back-off)

$$\hat{r}^{kNN}_{uj} = \mu + b_u + b_j + \frac{\sum_{i\in N_k(j;u)} \tilde{s}_{ij}(r_{ui} - \mu - b_u - b_i)}{\sum_{i\in N_k(j;u)} \lvert\tilde{s}_{ij}\rvert}.$$

Cold-Start Fallback

$$\hat{r}_{uj} = \begin{cases} \hat{r}^{kNN}_{uj} & N_k(j;u) \neq \varnothing, \\ \mu + b_u + b_j & \text{no neighbours}, \\ \mu + b_j & \text{new user}, \\ \mu + b_u & \text{new item}, \\ \mu & \text{both new}. \end{cases}$$

Matrix Factorisation (SVD of Residuals)

$$R'_{ui} = \begin{cases} r_{ui} - (\mu + b_u + b_i), & (u,i) \in \Omega, \\ 0, & \text{otherwise}, \end{cases} \qquad R' \approx U_f \Sigma_f V_f^\top,$$

$$\hat{r}^{\text{MF}}_{ui} = \mu + b_u + b_i + \left(U_f \Sigma_f^{1/2}\right)_{u\cdot} \cdot \left(V_f \Sigma_f^{1/2}\right)_{i\cdot}.$$

Decision Tree (Regression Target)

orig: $y = r_{ui}$, res: $y = r_{ui} - (\mu + b_u + b_i)$, $\hat{r}^{\text{res}}_{ui} = (\mu + b_u + b_i) + f_\theta(x_{ui})$.

Prediction & Ranking Metrics

$$\text{RMSE} = \sqrt{\frac{1}{\lvert T\rvert}\sum_{(u,i)\in T} (r_{ui} - \hat{r}_{ui})^2}, \quad \text{MAE} = \frac{1}{\lvert T\rvert}\sum_{(u,i)\in T} \lvert r_{ui} - \hat{r}_{ui}\rvert,$$

$$\text{P@}k_u = \frac{\lvert \text{Top}k_u \cap \text{Rel}_u\rvert}{k}, \quad \text{R@}k_u = \frac{\lvert \text{Top}k_u \cap \text{Rel}_u\rvert}{\lvert \text{Rel}_u\rvert},$$

$$\text{MRR} = \frac{1}{\lvert U'\rvert}\sum_{u\in U'} \frac{1}{\text{rank}_u}, \quad \text{nDCG@}k_u = \frac{\sum_{t=1}^{k} \frac{2^{\text{rel}_{u,t}} - 1}{\log_2(t+1)}}{\text{IDCG@}k_u},$$

$$\text{CatalogCov@}k = \frac{\left\lvert \bigcup_u \text{Top@}k(u)\right\rvert}{\lvert I\rvert}.$$

Paired Testing & Holm–Bonferroni

$$d_s = \text{RMSE}_s(A) - \text{RMSE}_s(B), \quad t = \frac{\bar{d}}{s_d/\sqrt{\lvert S\rvert}}, \quad \text{df} = \lvert S\rvert - 1.$$

Sort p-values $p_{(1)} \leq \cdots \leq p_{(m)}$, reject if

$$p_{(i)} \leq \frac{\alpha}{m - i + 1}, \qquad p^H_{(i)} = \max_{j\leq i}\left((m - j + 1)\, p_{(j)}\right).$$

Overlap / Weight Diagnostics

$$h(m) = \lvert\{(i,j) : n_{ij} = m\}\rvert, \qquad \mathbb{E}\left[\frac{n_{ij}}{n_{ij} + \alpha}\right] \text{ over pairs with } n_{ij} \geq \tau.$$

C | Tools Utilized

Core Python Libraries

- `numpy` — numerical arrays, linear algebra
- `pandas` — data frames, CSV I/O
- `scipy.sparse` — CSR matrices for ratings

Machine Learning (scikit-learn)

- `TruncatedSVD` — residual MF factors
- `DecisionTreeRegressor` — tree (orig/res targets)
- `metrics` — MAE/RMSE
- `model_selection` (basic splits)

Statistics

- `scipy.stats.ttest_rel` — paired t-tests

Parallel / Concurrency

- `multiprocessing` (*spawn*) — process pools for kNN similarity & seeds
- `concurrent.futures` — task fan-out for compute chunks

System / Profiling

- `psutil` — peak resident memory, CPU cores

Visualization

- `matplotlib` — runtime/memory/RMSE and comparison plots

Data Acquisition

- `kagglehub` — Goodbooks-10K ratings download (fallback)

Reproducibility Artifacts

- `env_spec.txt` — Python & package versions
- `metadata.json` — seeds, grids, worker settings

D | References

[1] Charu C. Aggarwal. *Recommender Systems: The Textbook*. Springer, 2016. Available at https://link.springer.com/book/10.1007/978-3-319-29659-3.

[2] Michael D. Ekstrand, John T. Riedl, and Joseph A. Konstan. Collaborative filtering recommender systems. *Foundations and Trends in Human–Computer Interaction*, 4(2):81–173, 2011. Foundational survey of collaborative filtering algorithms and evaluation methods.

[3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009. Available at https://web.stanford.edu/~hastie/ElemStatLearn/.

[4] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1):5–53, 2004. Framework for evaluation including RMSE and MAE.

[5] Zan Huang, Hsinchun Chen, and Daniel Zeng. Applying associative retrieval techniques to alleviate the sparsity problem in collaborative filtering. *ACM Transactions on Information Systems*, 22(1):116–142, 2004. Addresses data sparsity issues in collaborative filtering.

[6] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. Available at https://www.statlearning.com/.

[7] Jae Kyeong Kim, Yoon Ho Cho, Won Joo Kim, Jae Ryong Kim, and Jee Hyong Suh. A recommendation algorithm using multi-level association rules. In *Proceedings of the IEEE/WIC International Conference on Web Intelligence*, pages 621–624, 2003. Association rule based approach to recommendation.

[8] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. Influential paper on matrix factorization approaches including SVD.

[9] Miroslav Kubat. *An Introduction to Machine Learning*. Springer International Publishing, 2nd edition, 2017. Course textbook - covers fundamental ML concepts including k-NN, decision trees, and evaluation methods.

[10] Greg Linden, Brent Smith, and Jeremy York. Amazon.com recommendations: item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1):76–80, 2003. Scalable item-based collaborative filtering.

[11] Kunwoo Park, Meeyoung Cha, and Eunhee Rhim. Positivity bias in customer satisfaction ratings, 2018. arXiv preprint: RNN/LSTM/GRU models on Samsung live chat logs reveal under-reporting driven positivity bias.

[12] Fabian Pedregosa, Ga"el Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Primary machine learning library used for implementation.

[13] Real Python. Build a recommendation engine with collaborative filtering, 2022. Practical implementation guide for collaborative filtering in Python.

[14] Najlae Quadar, Hicham Chaibi, Abderrahim Chehri, Tejinder Sharma, and Rachid Saadane. A book recommender system using collaborative filtering method. In *International Conference on Data Science, E-learning and Information Systems 2021*, pages 1–6, 2021. Comparison of user-based, item-based CF and matrix factorization for books.

[15] GroupLens Research. Movielens datasets. University of Minnesota, 2023. Available at https://grouplens.org/datasets/movielens/.

[16] Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. Grouplens: An open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work*, pages 175–186, 1994. Available at https://dl.acm.org/doi/10.1145/192844.192905.

[17] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web*, pages 285–295, 2001. Seminal paper on item-based collaborative filtering using cosine similarity.

[18] Andrew I. Schein, Alexandrin Popescul, Lyle H. Ungar, and David M. Pennock. Methods and metrics for cold-start recommendations. In *Proceedings of the 25th Annual International ACM SIGIR Conference*, pages 253–260, 2002. Handling new users and items in recommendation systems.

[19] Xiaoyuan Su and Taghi M. Khoshgoftaar. A survey of collaborative filtering techniques. *Advances in Artificial Intelligence*, 2009, 2009. Available at https://www.hindawi.com/journals/aai/2009/421425/.

[20] Zygmunt Zajac. Goodbooks-10k dataset, 2017. Primary dataset source - 10,000 books, 6 million ratings.